

Chapitre 3: Les systèmes de gestion de version – partie 2

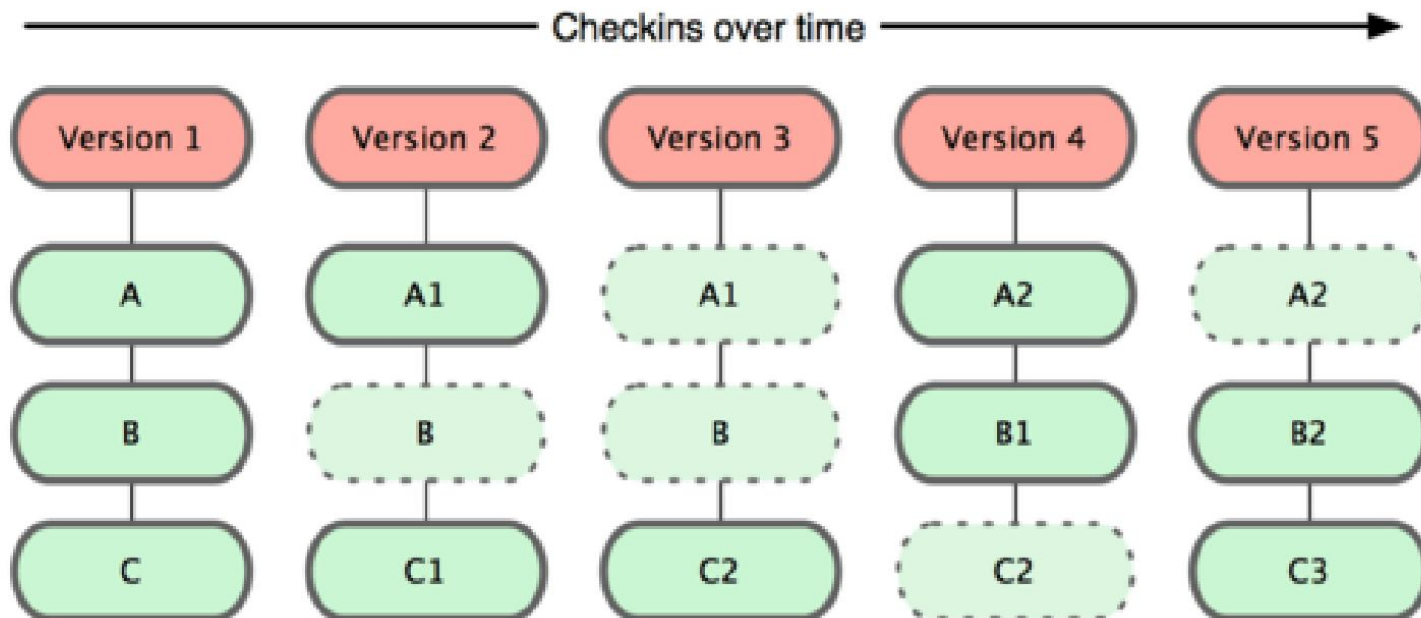
*Ce document doit être complété par les notes
de cours !*

GIT

- Système de gestion de version distribué (DVCS) 
- Développement lancé par Linus Torvalds en 2005 pour remplacer BitKeeper (propriétaire) dans l'hébergement du code de linux
- Depuis, Git héberge le développement du noyau Linux.

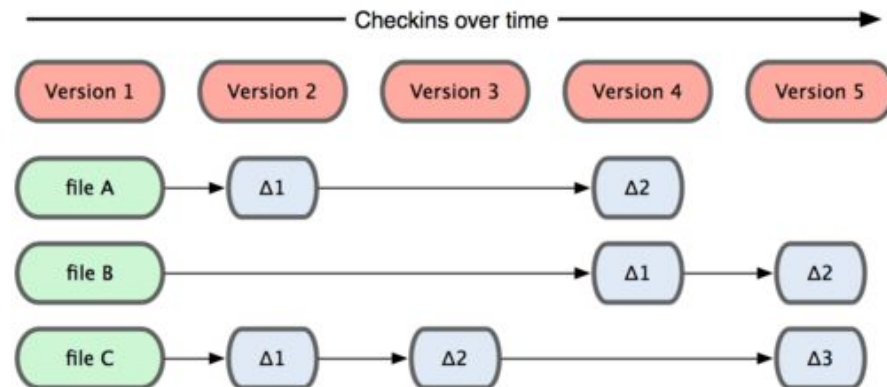
Principes de base

- Un dépôt Git
 - une sorte de système de fichiers (base de données),
 - enregistre les versions de fichiers d'un projet à des moments précis au cours du temps sous forme **d'instantanés (snapshot)**.



Principes de base (suite)

- À chaque fois qu'il y a validation l'état du projet dans Git
 - il prend effectivement un instantané du contenu de votre espace de travail à ce moment et enregistre une référence à cet instantané.
- Pour être efficace et si les fichiers n'ont pas changé,
 - Git ne stocke pas le fichier à nouveau
 - juste une référence vers le fichier original qui n'a pas été modifié.
- Fonctionnement différent la plupart des autres SGV
 - gèrent l'information comme une liste de modifications de fichiers

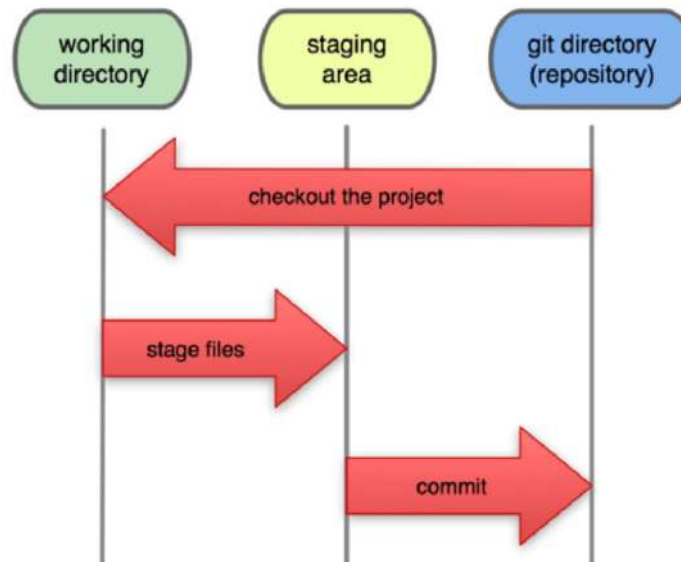


Principes de base (suite)

- 3 sections d'un projet Git :
 - Le répertoire Git (Git directory):
 - contient les méta-données et la base de données des objets du projet
 - Le répertoire de travail (working directory) :
 - extraction unique d'une version du projet depuis la base de données du dépôt.
 - La zone de transit/d'index (staging area) :
 - simple fichier contenant des informations à propos de ce qui sera pris en compte lors de la prochaine soumission.

Principes de base (suite)

- L'utilisation standard de Git :
 - L'utilisateur modifie des fichiers dans son répertoire de travail ;
 - Il indexe les fichiers modifiés
 - ce qui ajoute des instantanés de ces fichiers dans la zone d'index ;
 - Il valide
 - ce qui a pour effet de basculer les instantanés des fichiers de l'index dans la base de données du répertoire Git.

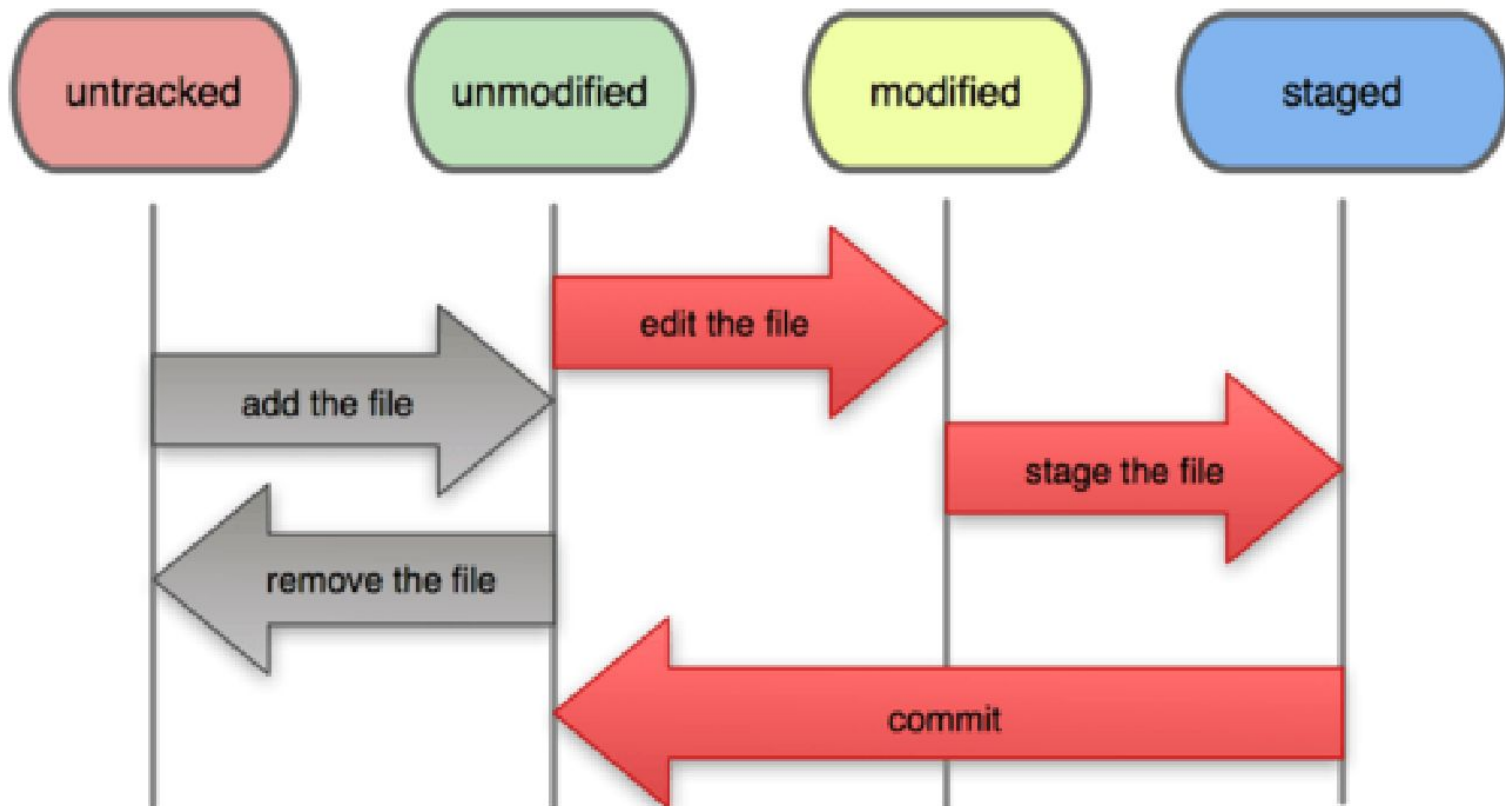


Principes de base (suite)

- 4 états d'un fichier dans Git :
 - **Non versionné** : fichier n'étant pas ou plus géré par Git ;
 - **Non modifié** : fichier sauvegardé de manière sûre dans sa version courante dans la base de données du dépôt ;
 - **Modifié** : fichier ayant subi des modifications depuis la dernière fois qu'il a été soumis ;
 - **Indexé (staged)** : idem, sauf qu'il en sera pris en instantané dans sa version courante lors de la prochaine soumission (commit)

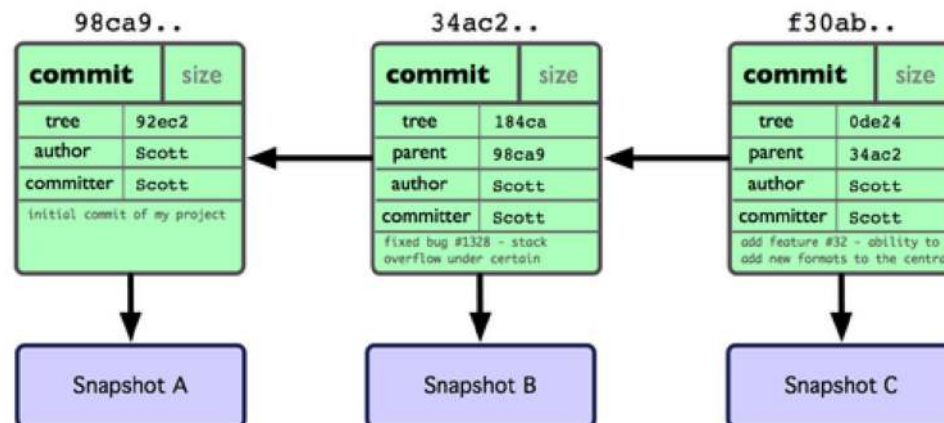
Principes de base (suite)

Cycle de vie du statut d'un fichier



Objet “commit”

- Chaque soumission (commit) crée un **objet “commit”**
 - contient un pointeur vers un instantané du contenu dont les modifications étaient indexées au moment de la soumission
 - ensemble d’objets permettant de stocker un instantané des fichiers concernés et de reproduire la structure du répertoire projet et de ses sous-répertoires
 - Des méta-données (auteur, message,...)
 - Un pointeur vers l’objet “commit” précédent.



Les commandes de base(07-02)

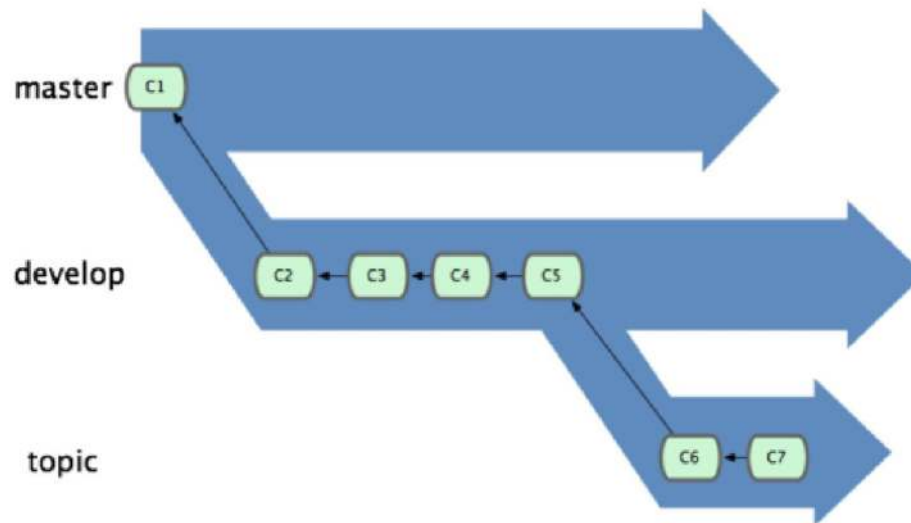
- Pour Initialiser un dépôt
 - **\$ git init**
- Pour afficher l'état des fichiers du répertoire courant
 - **\$ git status**
 - *Untracked files* : fichiers non versionnés.
 - *Changes to be committed* : modifications (ajout, suppression, changements) chargées en zone de transit (staging area), ou indexées.
 - *Changes not staged for commit* : modifications n'ayant pas été chargées en zone de transit (ou indexées).
- Pour indexer l'ajout ou les changements d'un fichier
 - **\$ git add <fichier>**
- Pour annuler les modifications indexées d'un fichier
 - **\$ git reset <fichier>**

Les commandes de base (suite)

- Pour annuler les modifications non encore indexées d'un fichier
 - **\$ git checkout [--] <fichier>**
- Pour indexer la suppression d'un fichier
 - **\$ git rm <fichier>**
- Pour Déversionner un fichier
 - **\$ git rm --cached <fichier>**
 - Garder le fichier dans le repertoire de travail mais l'enlever des fichier suivis par les versions (indexés)
 - L'opposé de **\$ git add <fichier>**
- Pour afficher le détail des modifications non indexées
 - **\$ git diff**
- Pour afficher le détail des modifications indexées
 - **\$ git diff --staged**
- Pour soumettre les modifications indexées en zone de transit
 - **\$ git commit**
- Pour voir l'historique des soumissions
 - **\$ git log**

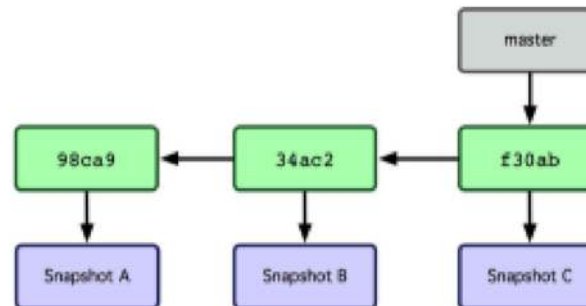
Branches

- Deux lignes d'évolution divergent de la ligne d'évolution courante
 - Les deux se poursuivent indépendamment l'une de l'autre.



Branches (suite)

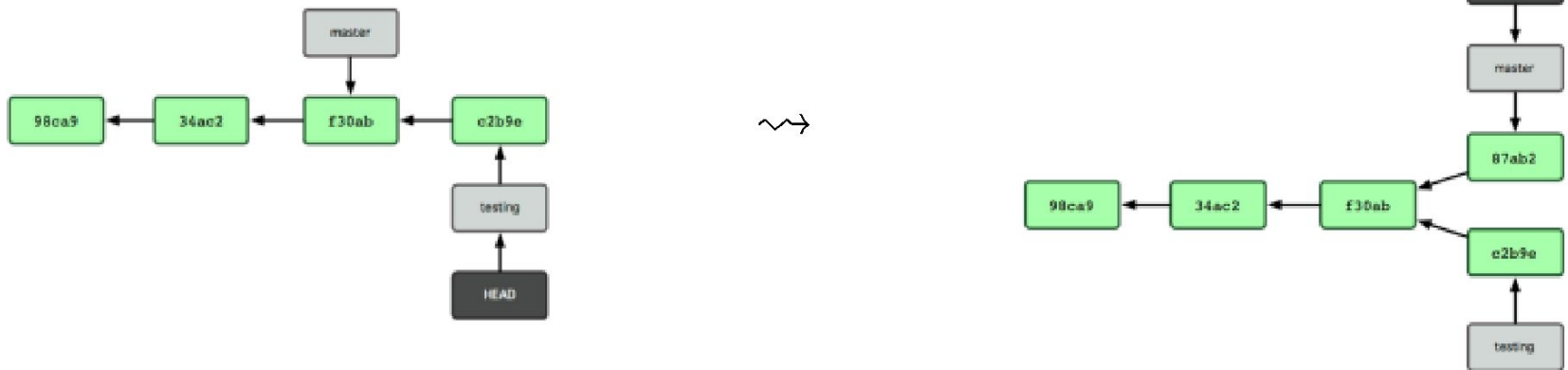
- Une branche dans Git est un pointeur vers un objet “commit”.
 - Par défaut, il existe une seule branche : **master**



- Pour créer une nouvelle branche
 - `$ git branch <branche>`
- **HEAD** est un pointeur spécial vers la branche sur laquelle on travaille actuellement (extraite dans le répertoire de travail).

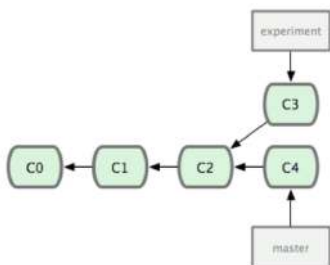
Branches (suite)

- Pour voir les branches du dépôt local
 - **\$ git branch**
- Pour supprimer une branche
 - **\$ git branch -d <branche>**
- Passer à une branche donnée
 - **\$ git checkout <branche>**
 - mise à jour de l'index, du répertoire de travail, et du pointeur HEAD

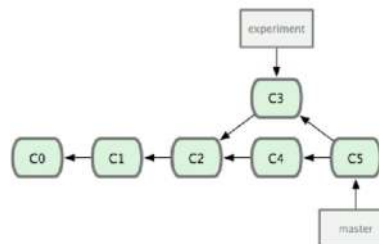


Branches (suite)

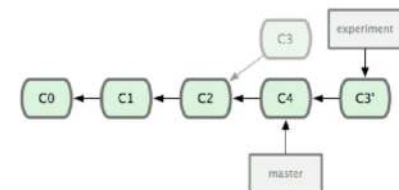
- Deux façons pour inclure les modifications d'une branche dans la branche courante:
 - **Merge**
 - une fusion à trois branches entre les deux derniers instantanés de chaque branche (C3 et C4) et l'ancêtre commun le plus récent (C2), créant un nouvel instantané (et un commit)
 - **Rebase**
 - prendre le patch de la modification introduite en C3 et le réappliquer sur C4



Fusionner (merge)

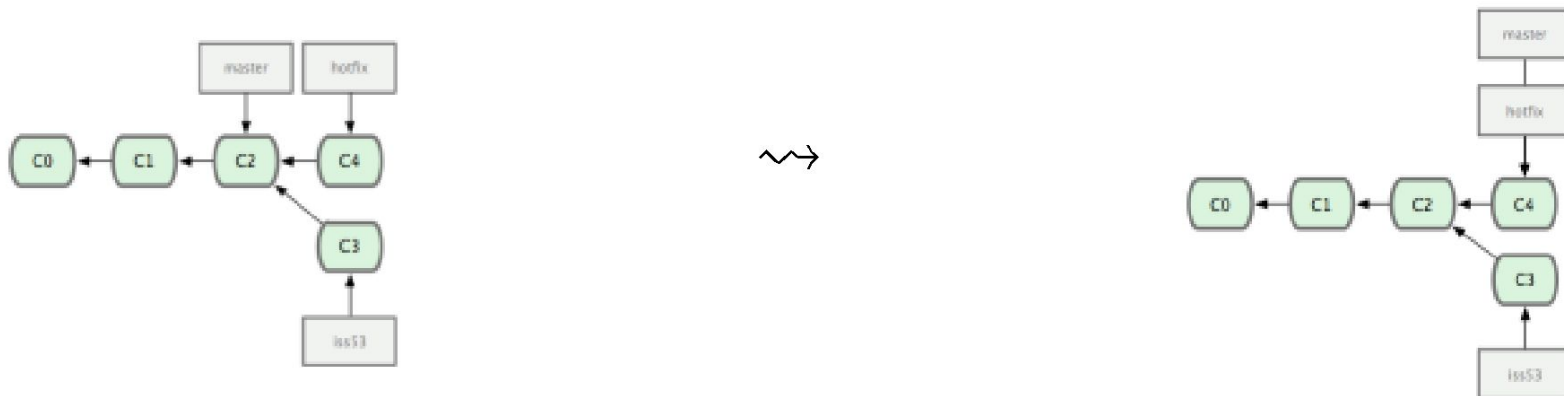


Rebaser (rebase)



Fusionner une branche

- Fusionner les modifications d'une branche donnée dans la branche courante (HEAD)
 - **\$ git merge <branche>**
 - Si l'objet "commit" pointé par <branche> est un descendant de l'objet "commit" courant,
 - seul le pointeur de la branche courante est déplacé sur l'objet "commit" concerné par la fusion ("fast-forward").



Fusionner une branche (suite)

- Git se base sur trois instantanés différents
 - celui de l'objet "commit" courant,
 - celui de l'objet "commit" pointé par <branche>
 - et celui de l'objet "commit" correspondant au plus jeune ancêtre commun des deux premiers objets "commit",
- pour créer un nouvel instantané associé à un nouvel objet "commit" par la fusion de deux branches, sur lequel la branche courante pointera à présent.



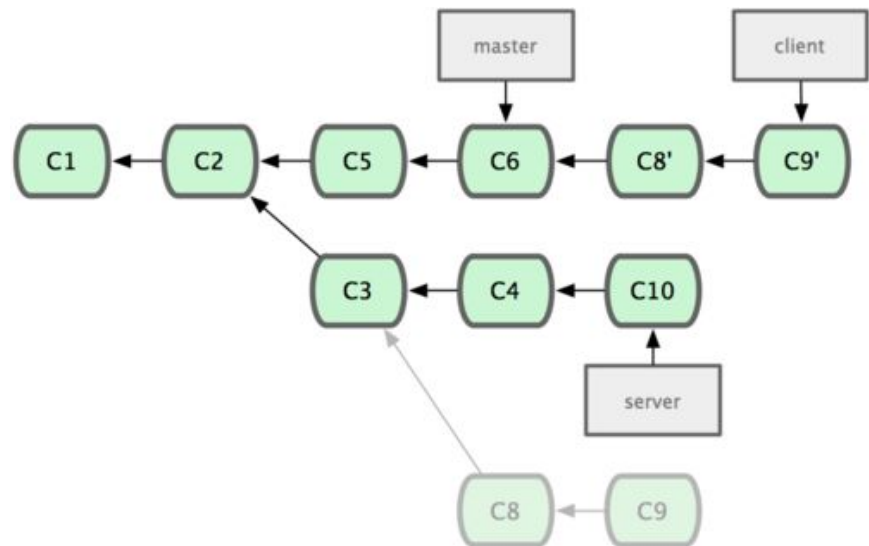
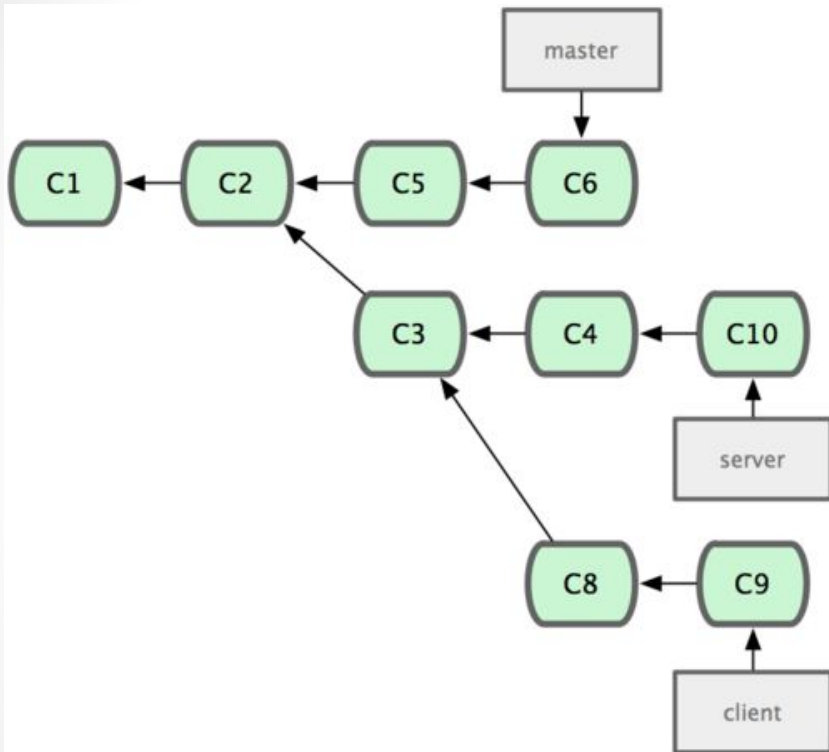
Fusionner une branche (suite)

- En cas de conflit empêchant la fusion
 - Aucun objet “commit” de fusion n’est créé
 - le processus est mis en pause.
 - **git status** : donne les fichiers n’ayant pas pu être fusionnés (listés en tant que “unmerged”).
- Git ajoute des marqueurs de résolution de conflits à tout fichier sujet à conflits pour qu’ils soit résolu à la main.
- Pour marquer les conflits dans un fichier <fichier> comme résolu
 - il faut utiliser la commande **git add <fichier>**
- Après résolution de tous les conflits, il est possible de soumettre les modifications sous forme d’objet “commit” avec **git commit**

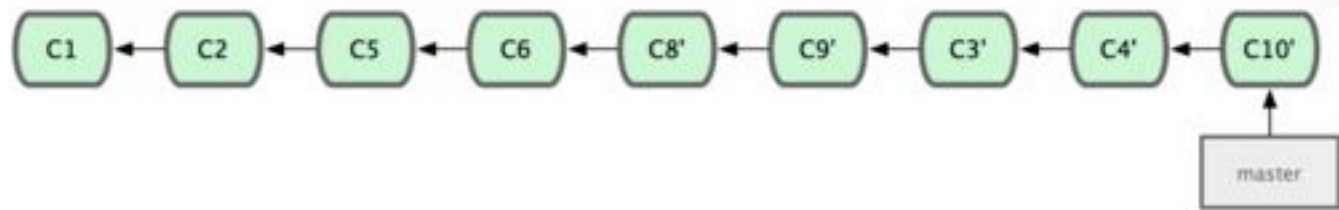
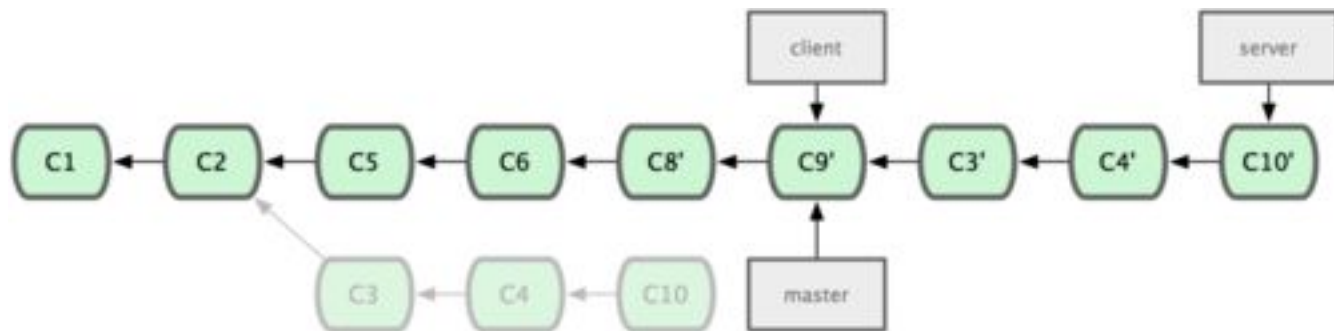
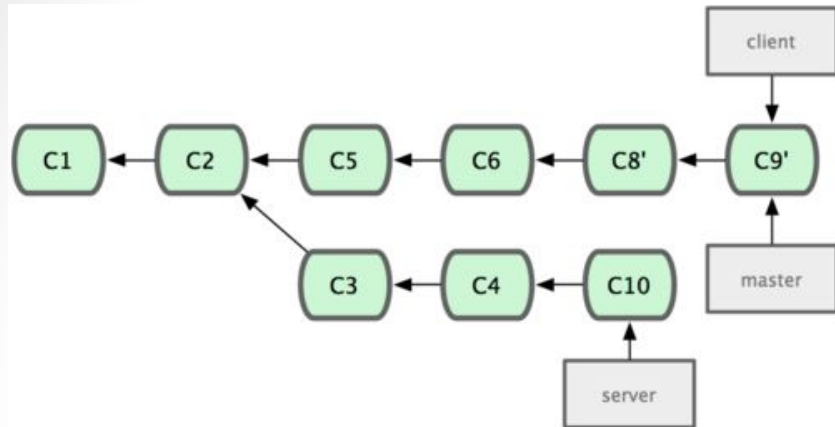
Rebaser une branche

- Rebaser les modifications de la branche courante (HEAD) sur une branche donnée
 - **\$ git rebase <branche>**
 - Aller à l'objet "commit" correspondant au plus jeune ancêtre commun des deux objets "commit" pointés par la branche courante et <branche>.
 - Obtenir et sauvegarder les changements introduits depuis ce point par chaque objet "commit" de la branche courante.
 - Faire pointer la branche courante sur le même objet "commit" que <branche> et ré-appliquer tous les changements un par un.

Rebaser une branche (suite)



Rebaser une branche (suite)

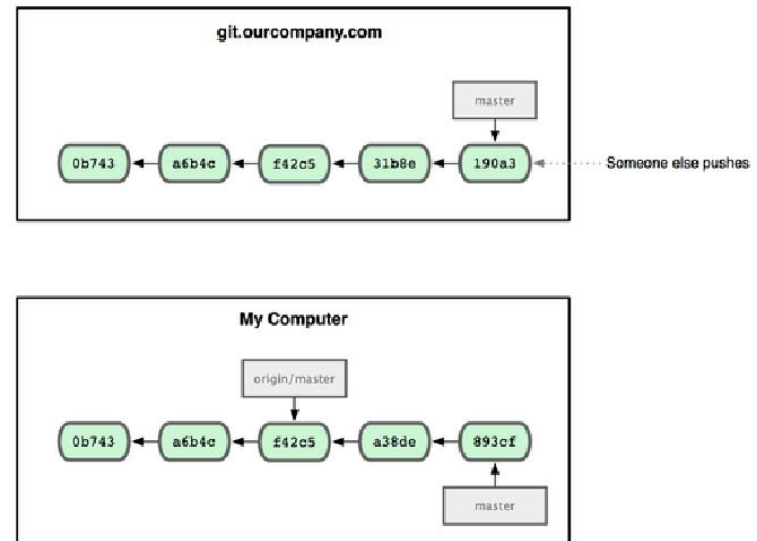
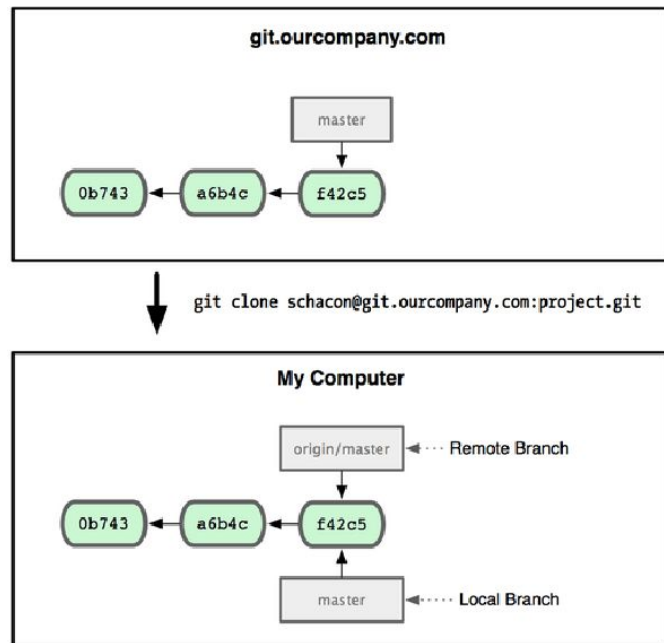


Travail avec des dépôts distants

- Echanger avec un ou plusieurs dépôts distants hébergeant le même projet
 - (typiquement des dépôts publics associés à une personne, une équipe ou tout le projet).
- Les données des dépôts distants (objets “commit” et instantanés) sont entièrement copiées dans le dépôt local
- Pour chaque branche <branche> d’un dépôt distant <dépôt> est maintenue une branche locale <dépôt>/<branche> non modifiable
 - permet de suivre la position de <branche> sur <dépôt> localement.
- Pour afficher la liste de tous les dépôts distants du projet
 - ***\$ git remote***

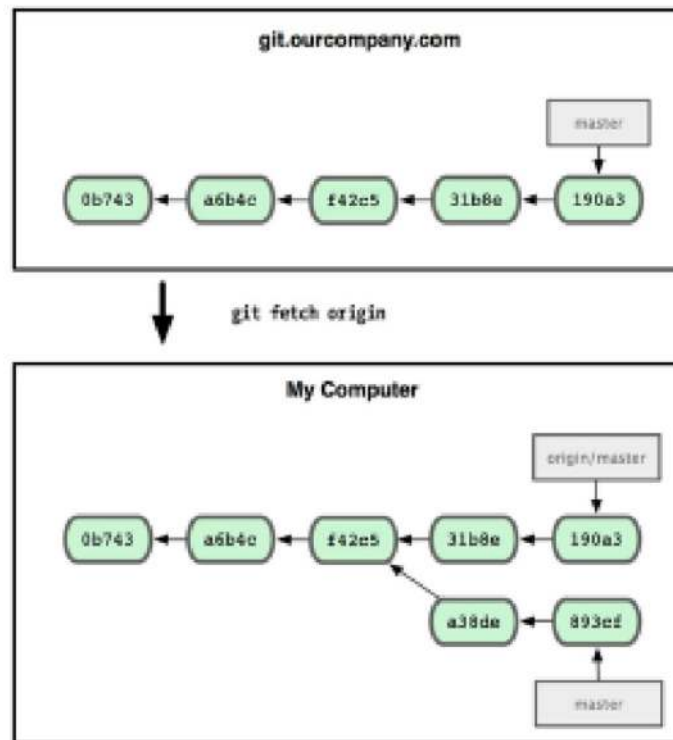
Travail avec des dépôts distants (suite)

- Cloner un dépôt distant (automatiquement nommé origin)
 - `$ git clone <URL> [<répertoire>]`



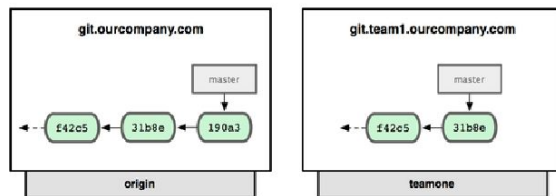
Travail avec des dépôts distants (suite)

- Pour Récupérer les modifications d'un dépôt distant
 - *\$ git fetch <dépôt>*

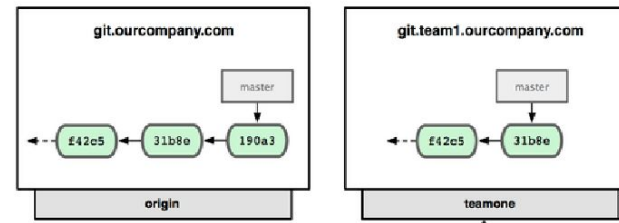
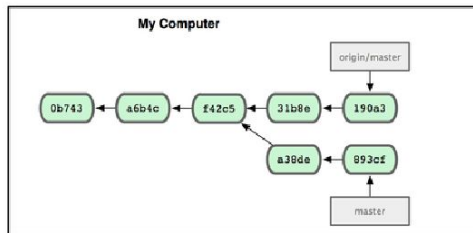


Travail avec des dépôts distants (suite)

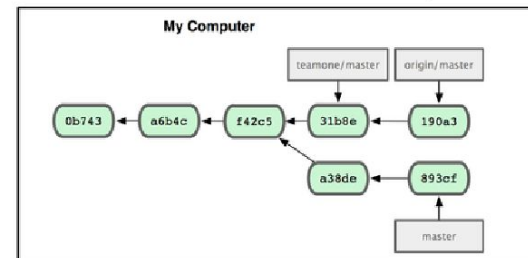
- Pour ajouter un dépôt distant
 - **\$ git remote add <nom> <URL>**



`git remote add teamone git://git.team1.ourcompany.com`



`git fetch teamone`



Travail avec des dépôts distants (suite)

- Pour mettre à jour un dépôt distant donné pour une certaine branche locale
 - ***\$ git push <dépôt> <branche>***
 - fonctionne uniquement si le dernier objet “commit” de la branche concernée du dépôt distant a été récupéré et intégré dans la branche en question du dépôt local
- Pour combiner git fetch et git merge
 - ***git pull <dépôt> <branche>***
- Pour combiner git fetch et git rebase
 - ***git pull --rebase <dépôt> <branche>***

GitHub

GitHub

- Offre un service web d'hébergement et de gestion de projets de développement logiciel basé sur Git.
- En plus , GitHub offre un éco-système complet qui inclut un Wiki et un système de tickets.
- <https://github.com>

Reference

- <https://git-scm.com>
- Documentation en français
 - <https://git-scm.com/book/fr/v2>